COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/28/26	**DATE SUBMITTED:**	09/02/26
NAME:	Cruz, Elijah J	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6 Searching Algorithms

1. PROCEDURE OUTPUT

SearchingAlgo.h
HEADER FILE

Contains the functions and logic that operate on the data fed by the user upon input. This includes both the different types of Linear and Binary Search, containing the functions linearSearchAR, linearSearchLL, binarySearchArr, binarySearchLL and getMiddle function that find the middle using the slow and fast pointer so that the binarySearchLL knows when to split the list. While the other 2 functions use loops in order to check indexes however, they are running under different conditions.

LINEAR SEARCH (SORTED AND UNSORTED LISTS)

```
/* LINEAR SEARCH ALGORITHM
*/

template <typename T>
void linearSearchAR(int size, T dataFind, T array[]) {
  //while loop
  int i = 0;
  while (i < size) {
     if (dataFind == array[i]) {
        std::cout << "searching is successful\n";
        return;
     }
     i++;
  }
  std::cout << "searching is unsuccessful\n";
  /*
  //for loop
  for (int i = 0; i < size; i++) {
     if (dataFind == array[i]) {
        std::cout << "searching is successful\n";
        return;
     }
  }
  std::cout << "searching is unsuccessful\n";
  */
}

template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind) {
  //repeat the loop until the current node is not NULL

  while (nodes != nullptr) {

     //check if the item = current node data
     if (nodes->data == dataFind) {
        std::cout << "searching is successful\n";
        return;
```

```
    }
    //move on to the next node after
    nodes = nodes->next;
  }
  //display unsuccessful
  std::cout << "searching is unsuccessful\n";
}
/*
```

BINARY SEARCH (SORTED ARRAYS)

```
/*
BINARY SEARCH ALGORITHM
*/

template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
  int low = 0, high = arrSize - 1, middle;
  while (low <= high) {
    middle = (low + high) / 2;
    if (dataFind == data[middle]) {
      std::cout << "\ndata was found\n";
      return;
    }
    else if (dataFind < data[middle]) {
      high = middle - 1;
    }
    else {
      low = middle + 1;
    }
  }
  std::cout << "data was not found\n";
}

//middle function getter:
/*
Slow variable moves by one in the loop while fast moves two while
in a loop going through each letter in the name JIMLORD
*/
template <typename T>
Node<T>* getMiddle(Node<T> *start, Node<T> *last) {

  Node<T> *slow = start;
  Node<T> *fast = start;

  //continue until fast reaches the end or the last one of the current search range
  while (fast != last && fast->next != last) {
    //move slow one step
    slow = slow->next;
    //move fast two steps
    fast = fast->next;
    if (fast != last) fast = fast->next;
  }
  return slow;
}

template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
  //step one: set start to the head of the list and set last to null
  Node<T>* last = nullptr;

  //if the middle element is same as key to be searched, we return
  while (start != last) {
    Node<T>* middle = getMiddle(start, last);
    if (middle->data == dataFind) {
      std::cout << "searching is successful\n";
      return;
    }
    //if middle element is less than the key, search the right half
    else if (middle->data < dataFind) {
```

```
        start = middle->next;
    }
    //if middle element is greater than the key, search the left half
    else {
        last = middle;
    }
  }
  std::cout << "searching is unsuccessful\n";
}
```

Node.h
HEADER FILE

Data structure file that defines the class Node and creates Nodes that are pointed at to store and allocate data.

```
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node {
public:
  T data;
  Node* next;
};

template <typename T>
Node<T> *newNode(T newData) {
  Node<T> *newNode = new Node<T>;
  newNode->data = newData;
  newNode->next = nullptr;
  return newNode;
}

#endif
```

Main.cpp
CPP FILE

Cpp file responsible for generating test data and collecting user input for the different array-based searches. It also is responsible for running the searches and for generating the random 50 numbers while also creating and connecting the nodes to be used by these said searching algorithm functions.

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main () {
  int dataset[max_size];
  srand(time(0));
  for (int i = 0; i < max_size; i++) {
      dataset[i] = rand();
      std::cout << dataset[i] << " ";
  }
  std::cout << std::endl;
```

```
// user input that determines as well if searching is successful or not
std::cout << "what number to find: ";
int choice;

std::cin >> choice;
linearSearchAR(max_size, choice, dataset);

//Create a node for your name
Node<char> *name1 = newNode('J');
Node<char> *name2 = newNode('I');
Node<char> *name3 = newNode('M');
Node<char> *name4 = newNode('L');
Node<char> *name5 = newNode('O');
Node<char> *name6 = newNode('R');
Node<char> *name7 = newNode('D');

//CONNECT THE NODES
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

//linearSearchLL(name1, 'D');
// Linear searching

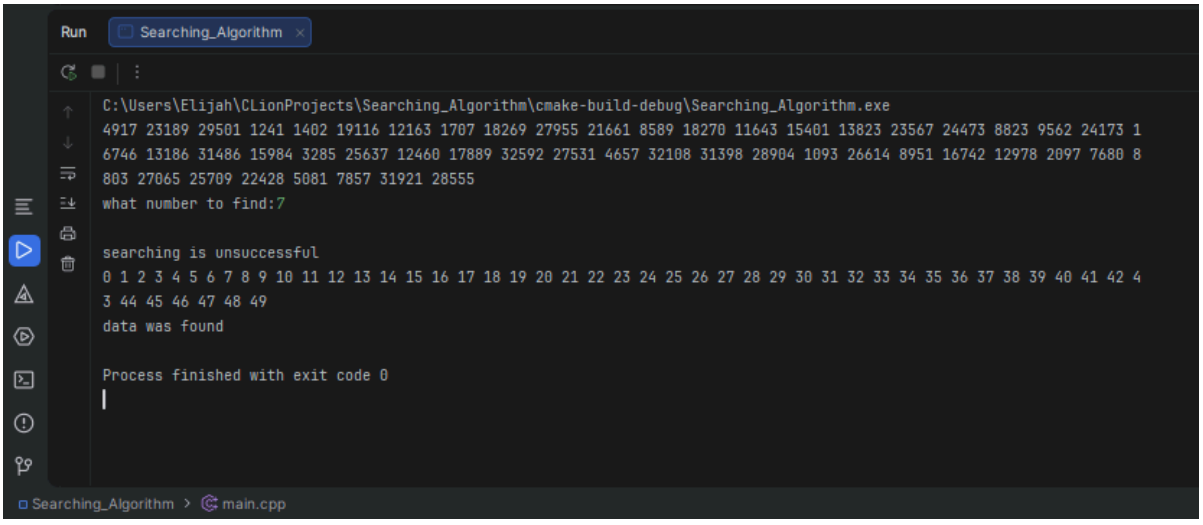
//generate a sorted array (use for loop)
int newArr[max_size] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

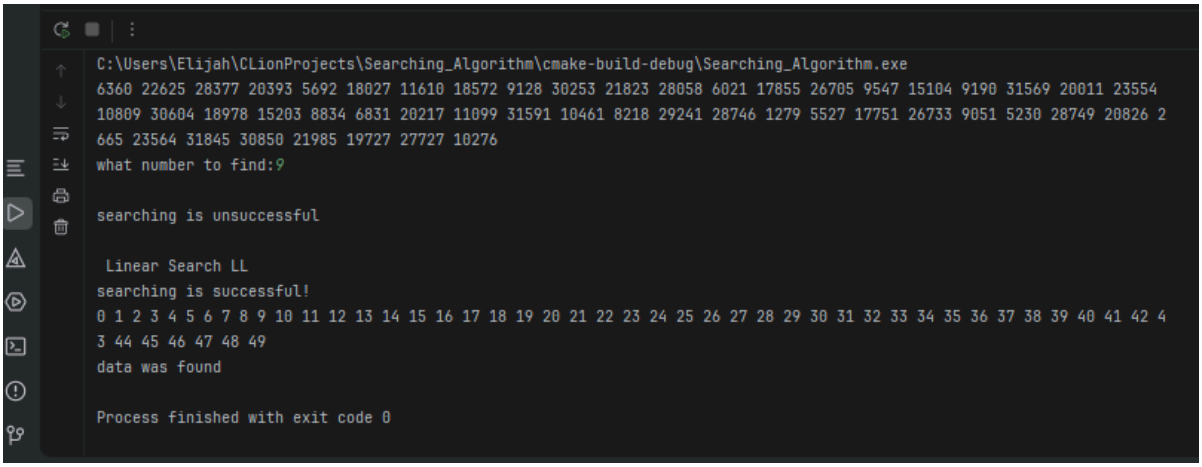
for (int i = 0; i < max_size; i++) {
    newArr[i] = i;
    std::cout << newArr[i] << " ";
}
binarySearchArr(newArr, 10, 4);

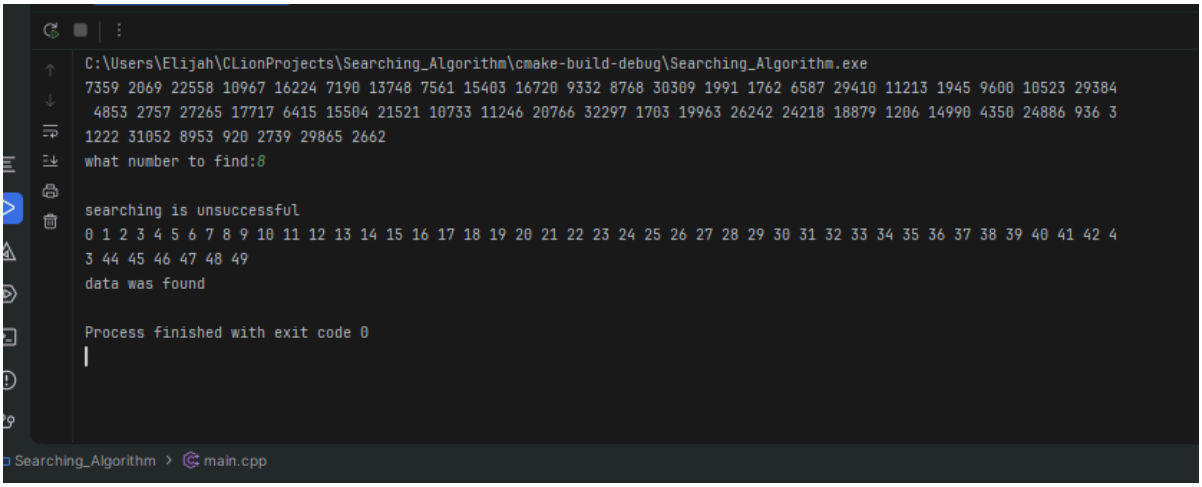
}
```

OUTPUT

LinearSearchArr





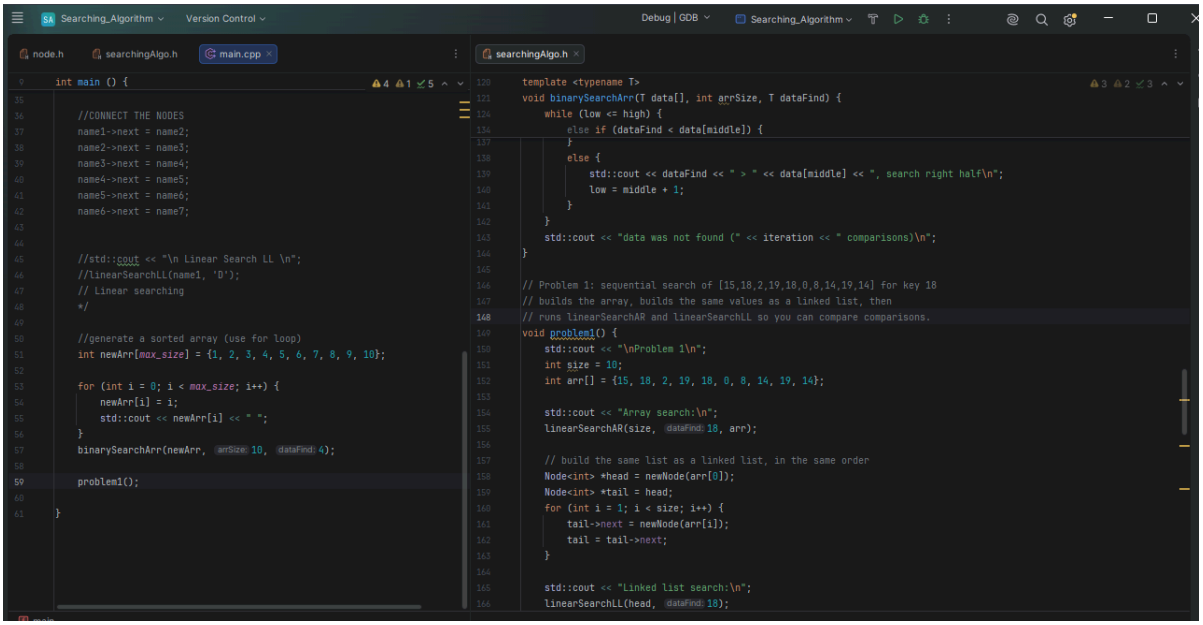


(JIMLORD block commented out)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

CODE



```
int main () {

    //CONNECT THE NODES
    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;


    //std::cout << "\n Linear Search LL \n";
    //linearSearchLL(name1, 'D');
    // Linear searching
    */

    //generate a sorted array (use for loop)
    int newArr[max_size] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    for (int i = 0; i < max_size; i++) {
        newArr[i] = i;
        std::cout << newArr[i] << " ";
    }
    binarySearchArr(newArr, arrSize: 10, dataFind: 4);

    problem1();

}
```

```
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
    while (low <= high) {
        else if (dataFind < data[middle]) {
        }
        else {
            std::cout << dataFind << " > " << data[middle] << ", search right half\n";
            low = middle + 1;
        }
    }
    std::cout << "data was not found (" << iteration << " comparisons)\n";
}

// Problem 1: sequential search of [15,18,2,19,18,0,8,14,19,14] for key 18
// builds the array, builds the same values as a linked list, then
// runs linearSearchAR and linearSearchLL so you can compare comparisons.
void problem1() {
    std::cout << "\nProblem 1\n";
    int size = 10;
    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

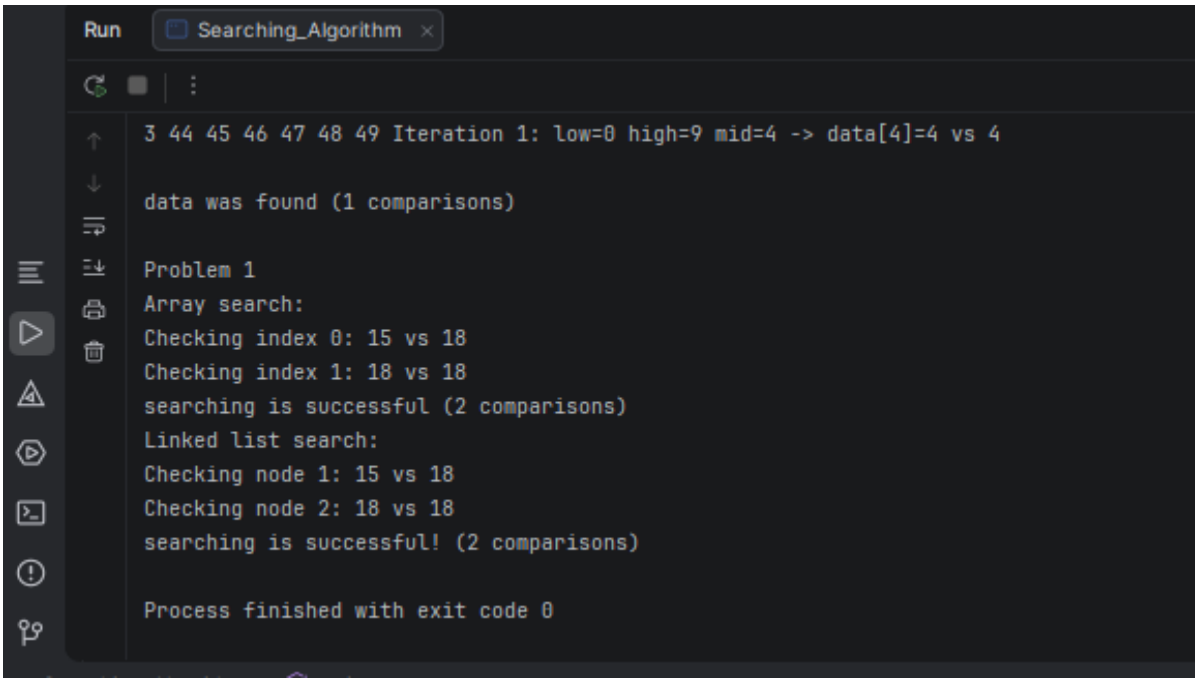
    std::cout << "Array search:\n";
    linearSearchAR(size, dataFind: 18, arr);

    // build the same list as a linked list, in the same order
    Node<int> *head = newNode(arr[0]);
    Node<int> *tail = head;
    for (int i = 1; i < size; i++) {
        tail->next = newNode(arr[i]);
        tail = tail->next;
    }

    std::cout << "Linked list search:\n";
    linearSearchLL(head, dataFind: 18);
```

This snippet shows the section of the code containing the main functions for both Linear Linked List and Array search for the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Both work in the similar manner, using While or For loops that iterate through each array and nodes linearly until the condition is met (dataFind == array[i] & nodes->data == dataFind) for each Linear search.

OUTPUT



```
Run   Searching_Algorithm

3 44 45 46 47 48 49 Iteration 1: low=0 high=9 mid=4 -> data[4]=4 vs 4

data was found (1 comparisons)

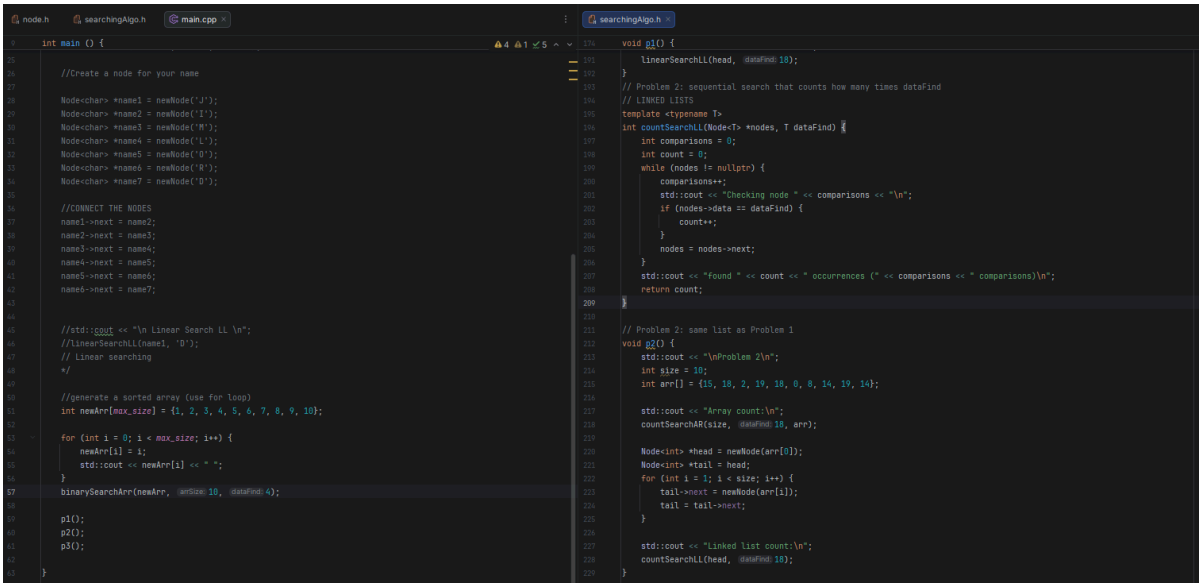
Problem 1
Array search:
Checking index 0: 15 vs 18
Checking index 1: 18 vs 18
searching is successful (2 comparisons)
Linked list search:
Checking node 1: 15 vs 18
Checking node 2: 18 vs 18
searching is successful! (2 comparisons)

Process finished with exit code 0
```

The output contains the results of using both Linear Linked List and Array search for the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14] and prints out the checked nods and the number of comparisons it took to find the key "18" within the list.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

CODE



```
int main () {

    //Create a node for your name

    Node<char> *name1 = newNode('J');
    Node<char> *name2 = newNode('I');
    Node<char> *name3 = newNode('M');
    Node<char> *name4 = newNode('L');
    Node<char> *name5 = newNode('O');
    Node<char> *name6 = newNode('R');
    Node<char> *name7 = newNode('D');

    //CONNECT THE NODES
    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;


    //std::cout << "\n Linear Search LL \n";
    //linearSearchLL(name1, 'D');
    // Linear searching
    */

    //generate a sorted array (use for loop)
    int newArr[max_size] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    for (int i = 0; i < max_size; i++) {
        newArr[i] = i;
        std::cout << newArr[i] << " ";
    }
    binarySearchArr(newArr, arrSize: 10, dataFind: 4);

    p1();
    p2();
    p3();

}
```

```
void p1() {
    linearSearchLL(head, dataFind: 18);
}
// Problem 2: sequential search that counts how many times dataFind
// LINKED LISTS
template <typename T>
int countSearchLL(Node<T> *nodes, T dataFind) {
    int comparisons = 0;
    int count = 0;
    while (nodes != nullptr) {
        comparisons++;
        std::cout << "Checking node " << comparisons << "\n";
        if (nodes->data == dataFind) {
            count++;
        }
        nodes = nodes->next;
    }
    std::cout << "Found " << count << " occurrences (" << comparisons << " comparisons)\n";
    return count;
}

// Problem 2: same list as Problem 1
void p2() {
    std::cout << "\nProblem 2\n";
    int size = 10;
    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

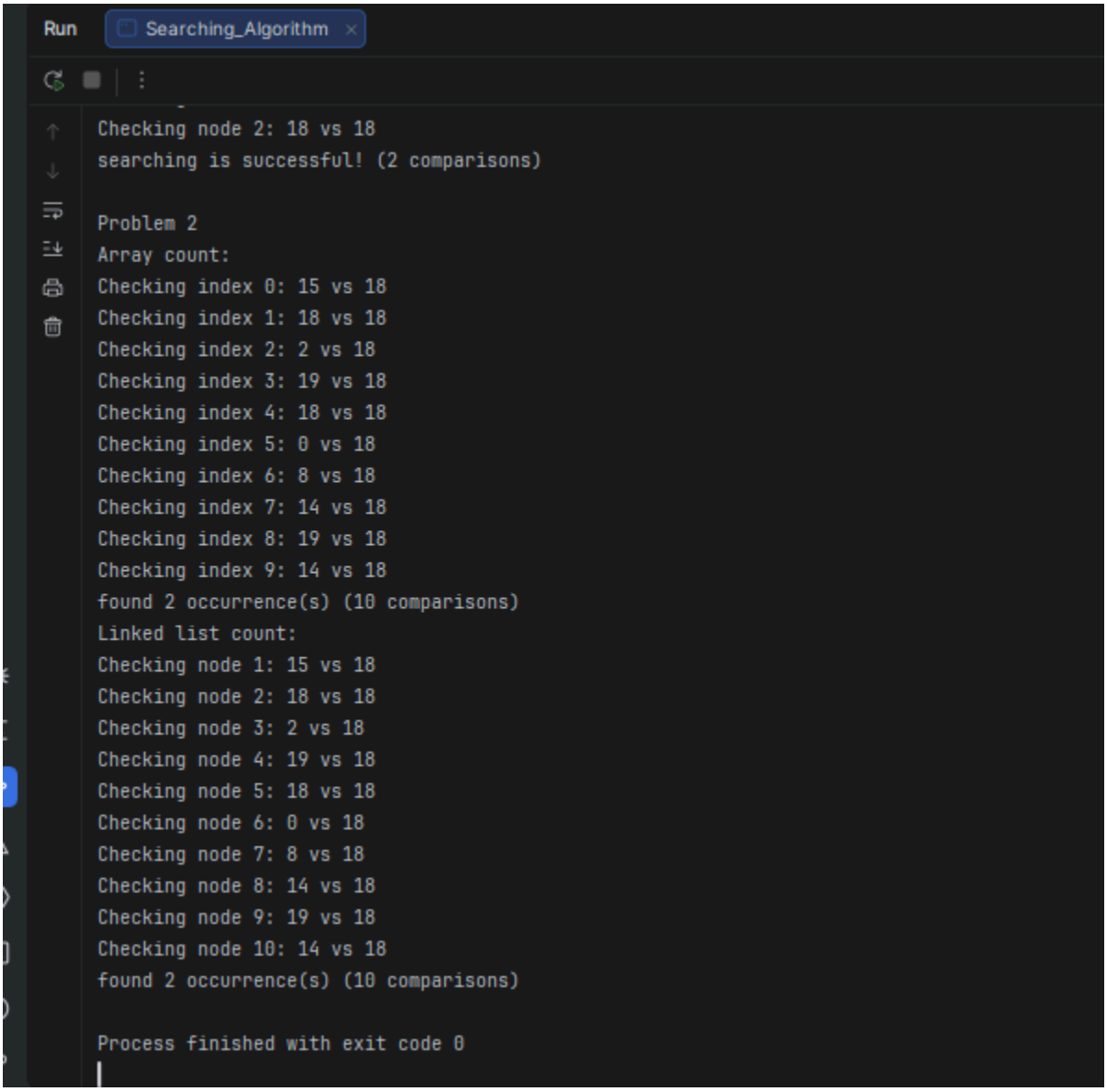
    std::cout << "Array count:\n";
    countSearchAR(size, dataFind: 18, arr);

    Node<int> *head = newNode(arr[0]);
    Node<int> *tail = head;
    for (int i = 1; i < size; i++) {
        tail->next = newNode(arr[i]);
        tail = tail->next;
    }

    std::cout << "Linked list count:\n";
    countSearchLL(head, dataFind: 18);
}
```

This snippet shows the section of the code containing the main functions for both the Array and Linked List count search (countSearchAR` and `countSearchLL) for the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Both work in a similar manner, using a For or While loop that iterates through the entire array and linked list, incrementing a `count` variable each time the condition is met (`dataFind == array[i]` for the array, `nodes->data == dataFind` for the linked list), without ever exiting early, so that all occurrences of the key (18) are counted instead of stopping once one is found.

OUTPUT



The output demonstrated how the modified sequential search counts the repeating instances of the key instead of stopping at the first match of the set condition. Both the Array and Linked List count search (countSearchAR and countSearchLL) check each index from 0 to 9 and nodes 1 to 10 as seen. Returning a value of 2 occurrences and 10 comparisons for each LinearSearch and we can observe that despite the structural differences with their code, the output and function can remain the same.

Problem 3 - 4. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you want to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

CODE

```
// problem 3 modified
template <typename T>
void binarySearchArr(T data[],int arrSize,T dataFind){
    int low = 0;
    int high = arrSize - 1;
    int middle;

    while (low <= high){
        middle = (int)((low+high)/2);
        std::cout << "low=" << low << " high=" << high << " middle=" << middle << " data[middle]=" << data[middle] << "\n";
        if (dataFind == data[middle]){
            std::cout << "The data was found: " << dataFind;
            return;
        }
        else if (dataFind < data[middle]){
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "The data was not found\n";
}
```

```
// Problem 3: binary search of sorted list [3,5,6,8,11,12,14,15,17,18]
void p3() {
    std::cout << "\nProblem 3\n";
    int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;

    binarySearchArr(arr, size, dataFind: 8);
}

#endif //SEARCHING_ALGORITHM_NODE_H
```

This snippet shows the section of the code containing the modified binary search that can print the execution of this Array binary search for finding the key "8" within the given list of [3, 5, 6, 8, 11, 12, 14, 15, 17, 18]. Both snippets of the code as seen contain the loop, if and else statements, and the mathematical operation of middle = (int)((low+high)/2).

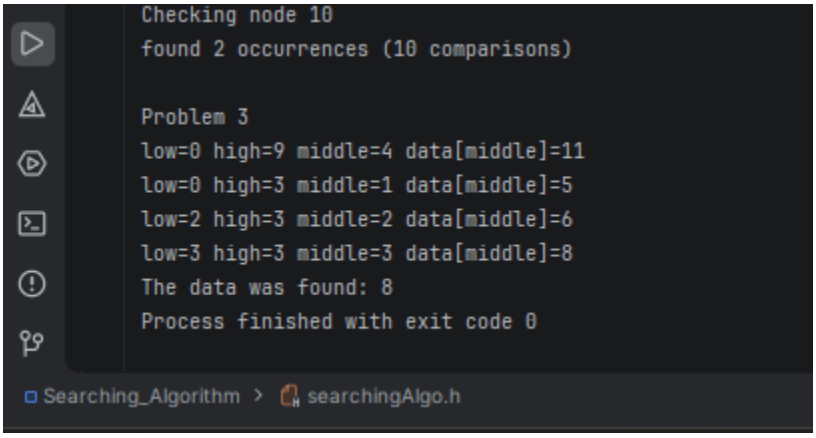
DIAGRAMS

ITERATION	0	1	2	3	4	5	6	7	8	9	OUTPUT
1	3	5	6	8	11	12	14	15	17	19	middle = (0+9)/2 = 4 -> data[4]=11 8 < 11, go left
2	3	5	6	8	11	12	14	15	17	19	middle = (0+3)/2 = 1 -> data[1]=5 8 > 5, go right
3	3	5	6	8	11	12	14	15	17	19	middle = (2+3)/2 = 2 -> data[2]=6 8 > 6, go right
4	3	5	6	8	11	12	14	15	17	19	middle = (3+3)/2 = 3 -> data[3]=8 The data was found



A Diagram showing how BinarySearch works within a sorted list, "removing" sections of the list and moving the low, middle, and high indexes until the DataFind is equivalent to the data[middle] meaning that the key was found. Thus, printing the data was found at the output.

OUTPUT



A snippet of the output of the code containing the values for low, middle, high and data[middle] indexes. This also shows that the data was found.

CONCLUSIONS

Summary of lessons learned

- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

This activity demonstrates and teaches students how both different types of a Linear and Binary search functions. As well as, how to integrate, apply, and use them based on the given requirements and conditions with different situations set. With the procedure focusing on visualizing how the binary and linear search works with linked lists and sorted arrays. With the supplementary focusing more on how to apply them with the given scenarios which include recursion and modification of the code with different lists. Problem 1, sequential search on both an array and a linked list needed only 2 comparisons to find the key 18, since it could stop as soon as a match was found. Problem 2 showed that modifying the algorithm to count all repeating instances removes that shortcut entirely, forcing it to check every element (10 comparisons) since a match doesn't guarantee there are no more later in the list. While problem 3 demonstrated the advantage of binary search on a sorted list, finding the key 8 in just 4 comparisons by repeatedly cutting the search range in half using the low, middle, and high pointers, instead of checking elements one by one. Overall, this gives us a conclusion that binary search is far more efficient on sorted data but can only work with the certain condition that the given datas are sorted and showing how each method arrives at its result.